

From push to position

Beyond Divar: command and location paths in Balad, Neshan, Snapp and Tapsi; what changed in sampled Android APKs, and what the files cannot tell us.

19 FEB

Neshan VT submission

2026

28 FEB

Iran war begins

2026

10 JUN

Balad uploader observed

2026

The Neshan and Balad dates are exact-file VT first submissions; the Neshan archive lists the same build on 18 February. Dates do not establish rollout, device use or a shared operator.

What the four-app review found

A companion to the Divar case file, based on acquired APKs rather than a claim about every release or device.

After Ramin Farajpour Cami's [Divar finding](#) and [my subsequent disclosure](#), I asked a narrower question: did contemporaneous Balad, Neshan, Snapp or Tapsi updates carry a comparable inserted delivery route, or add a sensitive location path that could matter in a targeting operation? The [Divar technical report](#) now covers 282 exact signed files, including a push-to-stage route first observed in 2023 and a later HTTP-header entry. This companion examines **103 exact APKs across seven package tracks** for other app behavior. [01-03]

I found **no Divar-equivalent notification-to-loader chain** in the inspected Balad, Neshan, Snapp or Tapsi samples. The most important distinct finding is in Neshan: a new server-pull route can feed an older diagnostic/file-upload command without showing a notification. A separate native network path later accepts GPS-derived input; its plaintext destination and historical traffic remain unresolved. [07-09]

Balad, Snapp driver and Tapsi have location/radio request paths with plausible positioning or service uses. Their sampled code establishes what each client could send under its gates, not that any company enabled the path for particular users or that an attacker received the data. I recovered no cross-app device join, shared operator, hostile command, upload receipt or malicious patch. Signing continuity does not settle authorization. [07,10-11]

What the APKs establish

TRACK	ESTABLISHED IN SAMPLED CODE	STILL UNKNOWN
Balad / 32	In the 26-file Bazaar track, remote-gated radio/location upload appears by 4.81.1 ; 4.84.0 broadens source choice. Six Play APKs form a separate package/signing track.	Historical AppMeta settings, wire fields, recipients and receipts.
Neshan / 21	14.8.10 adds pull into PUSH_LOGGER dispatcher; later native code can POST observation-derived data.	Pull commands, upload logs, native host/path, effective config and traffic.
Snapp / 17	Driver 5.17.0 adds radio collection POST; passenger 8.39.0 adds notification polling classes.	AB settings, dynamic destination, receipt logs and passenger activation.
Tapsi / 33	Geolocation POST is callable; one driver 8.8.0 channel contains a patch updater.	Jan-May APKs, feature settings, requests and patch delivery.

INTERPRETATION

These findings describe **capability and sampled version deltas**. They do not establish malicious collection, joint targeting, operator identity or use during the war. A negative Divar-pattern scan is a narrow result, not whole-app clearance.

The 103-file boundary

Each tile below is one acquired exact APK. Color identifies a package track; it is not a benign or malicious verdict.

BALAD BAZAAR		26
BALAD PLAY		6
NESHAN		21
SNAPP PASSENGER		12
SNAPP DRIVER		5
TAPSI PASSENGER		20
TAPSI DRIVER		13

FILLED / VT FIRST-SUBMISSION DATE ON RECORD

OUTLINE / ARCHIVE DATE OR NO EXACT-FILE VT DATE

The normalized [103-file inventory](#) retains SHA-256, package, versionCode, signer fingerprint and date basis for every tile, with archive listing/upload dates kept separately when available. One signer can group a lineage; it cannot tell who approved its code.

Where the sample runs stop

Version deltas are bounded by the acquired files on either side, never by presumed release cadence.

TRACK	ACQUIRED SPAN	CRITICAL GAP OR CAVEAT
Balad / Bazaar	26 APKs; 4.76.3-4.84.1	4.78.0/4.78.1, 4.80.0 and 4.81.0 bytes missing; a transient path cannot be excluded.
Balad / Play	6 APKs; 4.77.0-4.83.2	Separate package/signer; 4.79.0 to 4.82.1 is a wider code gap.
Neshan	21 APKs; 14.7.9-14.13.0.6	Ten archived bases plus eleven VT same-version variants; March-May availability unproved.
Snapp / passenger	12 APKs; 8.37.0-8.42.2	8.39.0 version archives straddle 28 Feb; preserved hash is tied to the 2 Mar listing.
Snapp / driver	5 APKs; 5.13.0-5.18.0	5.14.0 and 5.16.0 missing; two distinct 5.17.0 install-permission variants.
Tapsi / passenger	20 APKs; 7.3.0-8.4.0	First submissions 12 Jun-30 Sep; January-May exact-package bytes absent.
Tapsi / driver	13 APKs; 7.13.6-8.9.0	First submissions 6 Jul-22 Sep; 8.8.0 has two distribution flavors.

How negative checks were used

The review scanned for the exact Divar class, the four-field trigger and calibrated relay/worker signatures, then followed the app-specific push, polling, location, updater and native call paths above. A renamed loader, server-only behavior, native-only implementation or unsampled release can escape those checks. The four apps therefore receive **bounded sampled-file conclusions**, not an all-version clean label.

DATE RULE

VirusTotal first submission dates the first known VT appearance of **one hash**. Archive dates usually describe a version listing or upload. Neither identifies the build date, rollout cohort, installation date, live configuration or historical use.

Code observations against the war

Solid markers are exact-file VT first submissions. The hollow Snapp marker is an archive listing, not an exact-hash VT date.



* The preserved Snapp 8.39.0 APK is from an APKPure listing dated 2 March. Uptodown lists the **version** on 25 February, but that earlier listing is not tied to this exact hash. The colored bands mark public conflict milestones: opening strikes on 28 February, an announced two-week ceasefire announced on 7 April (UN statement published 8 April), and a June peace deal reported by the UN on 15 June. Intermittent fire continued after April. [04-06,10]

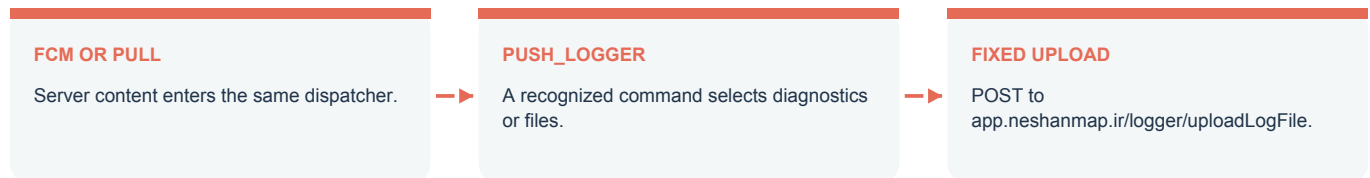
TIMING INFERENCE

Neshan's added pull path and Divar's returned route were in VT before 28 February. The acquired Balad, Snapp driver and Tapsi location-path checkpoints are from June or later. No acquired cross-app file or traffic proves pre-war location pre-positioning or coordinated deployment.

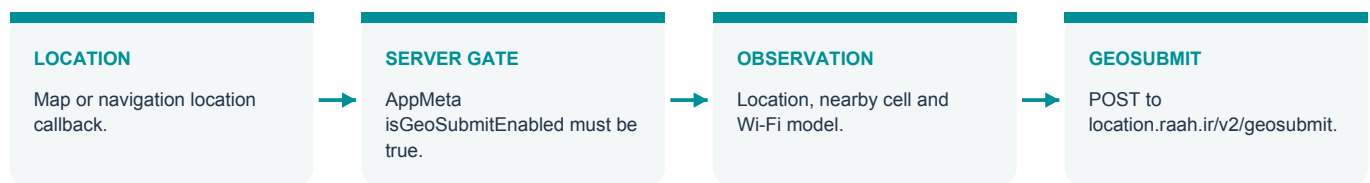
Distinct paths, no demonstrated join

Solid arrows summarize app-local code paths; dashed arrows rely on framework or server dispatch. Dotted lines below are an unverified cross-app hypothesis.

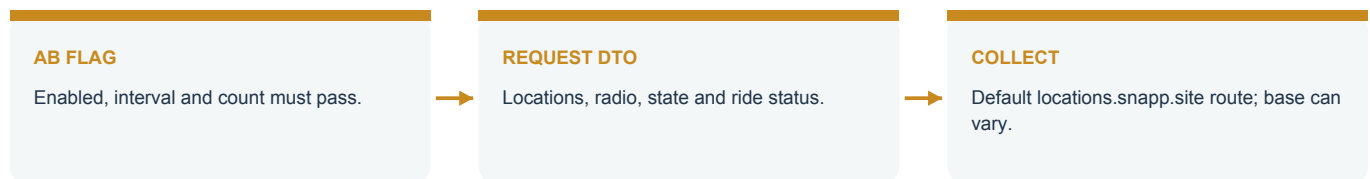
NESHAN / COMMAND INPUT



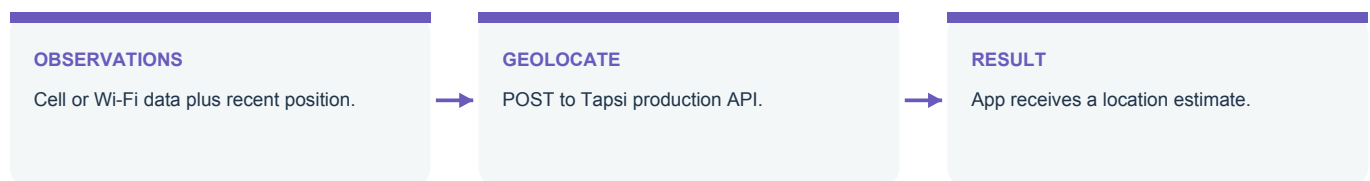
BALAD / RADIO OBSERVATION



SNAPP DRIVER / RADIO OBSERVATION



TAPSI / POSITIONING



HYPOTHESIS / NO LINK OBSERVED

If an operator held device-level location records in one service and targeting access in another, the first could inform a later payload send. No common device key, target list, operator, push receipt or second stage connects these APKs to Divar. App-local UUID hashes are not shown to be cross-app identifiers.

Figure 1. App-local paths are drawn separately. Solid is inspected code flow, dashed is framework/server delivery, and the dotted cross-app join remains a hypothesis. No sampled APK or traffic proves that join.

Three entries to one Neshan dispatcher

Reduced app-local graph from sampled 14.8.10 and 14.13.0.6 paths. The command branch predates both new pull entries.

ENTRY ROUTES

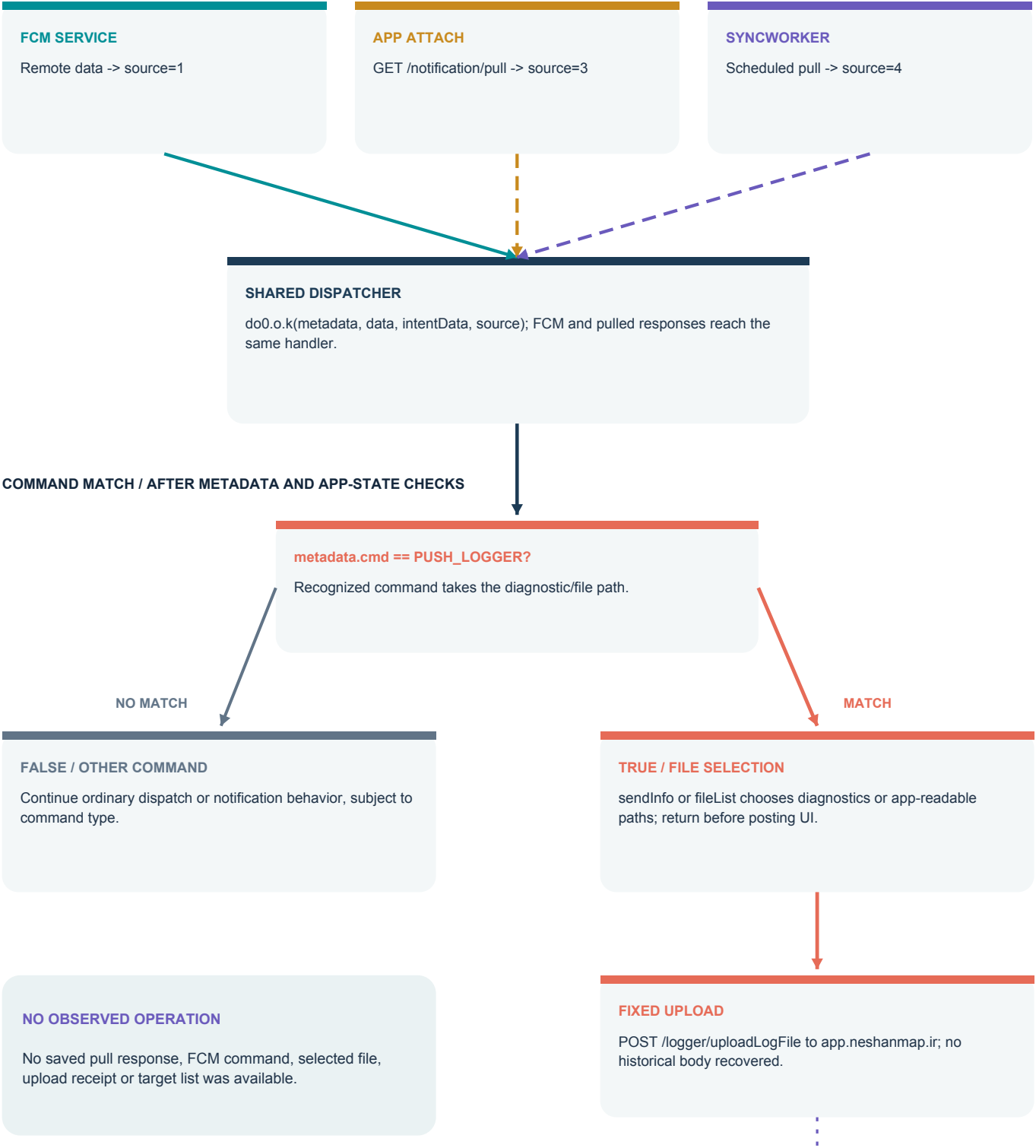


Figure 2. Solid edges are inspected app branches; dashed entry edges depend on Android/WorkManager or server scheduling. The dotted terminal marks missing operational evidence, not a demonstrated upload. The file-selection concern does not establish a historical attack.

A new path to an older command

The material change is ingress: a pulled server response reaches the same dispatcher as FCM.

In acquired **14.7.9** (archive upload 10 November 2025), Neshan already recognized **PUSH_LOGGER**. That command can package diagnostic data or selected app-readable files and upload them to app.neshanmap.ir/logger/uploadLogFile. It predates the 2026 war and is not itself a newly inserted Divar-style stage loader. [08]

Sampled **14.8.9** has no reviewed pull endpoint. In **14.8.10**, app attachment calls **GET /notification/pull**; the response's **metadata**, **data** and **intentData** enter the existing dispatcher with **source=3**. FCM calls that dispatcher with **source=1**. A recognized **PUSH_LOGGER** branch returns before ordinary notification posting, so the pull route does not require a visible notification or tap. The app adds a bearer header when it holds a token; the APK cannot reveal server-side access rules.

14.8.9 / SAMPLED BASELINE

```
FCM -> dispatch(command, source=1)
PUSH_LOGGER -> package / upload
no /notification/pull endpoint
```

14.8.10 / ADDED ENTRY

```
app attach -> GET /notification/pull
response -> dispatch(..., source=3)
PUSH_LOGGER -> same upload path
```

Normalized from the acquired DEX/JADX path. The new edge is a silent server pull, not a new executable downloader.

The later scheduled pull

By sampled **14.12.0.2**, **SyncWorker** also polls the endpoint. In the inspected **14.13.0.6** app, attachment seeds the work; the worker dispatches each response with **source=4** and schedules another run, including after a caught exception. The compiled **PullConfig** interval is eight hours, but server configuration and Android WorkManager can change effective timing. [08]

INSPECTED COMMAND CONTRACT / ILLUSTRATIVE FIELD NAMES

```
pull response: { metadata, data, intentData }
metadata.cmd == PUSH_LOGGER
data.sendInfo or data.fileList selects diagnostics / files
POST /logger/uploadLogFile -> app.neshanmap.ir
```

This is a normalized code contract, not a captured historical response or upload.

WHAT MAKES THIS AN OPEN LEAD

The selected-file helper concatenates command-supplied path parts without canonical containment in the inspected Java path. . . appears capable of selecting other files readable by the app. No historical traversal command, file contents or upload receipt was recovered.

Exact transition: **14.8.9** archive base [681b4f975d40b9d0f1dfa9d5d6b79d59cbd3ca46746a6f9f013ef1e1bf7bcfd4](#); **14.8.10** VT first submitted 19 February, [947c4a9412ef577b29c87d87764b4e9e0e3ddfd1966bff2cbab83adb3f2f8115](#). No manifest permission was added at this boundary. The missing March-May APKs limit when later changes began.

The opaque HTTPS branch evolved

A separate JNI path later accepts an observation; it is not connected to the reviewed push/pull dispatcher.

The earliest sampled *Impulse* bridge is **14.10.2.3**, first submitted to VT on 6 June. Its native *GetAll* path reaches HTTPS request code, but *nativeGetAll(long)* takes no Java observation String. The native object receives Android context and telephony state, so absence of that Java argument does not prove the request lacked location-related data. The exact body and runtime activity are unknown. [09]

By sampled **14.13.0.6**, Java serializes an observation that can include the last GPS fix and passes it to *nativeGetAll(long, String)*. The JNI function consumes the String, incorporates input-derived data, transforms it, and passes a body to a native HTTPS *POST* builder. This is stronger than a string hit: the observation reaches the request body at the static call-graph level. It does **not** establish which individual fields survive the merge or what any device sent. [09]

14.10.2.3 / JUNE nativeGetAll(handle) -> native state -> HTTPS helper no Java observation String arg	14.13.0.6 / LATER nativeGetAll(handle, obsJson) -> parse / merge / encrypt -> JSON POST body -> HTTPS
--	---

Semantic JNI comparison. No plaintext network capture was recovered, and the later GPS input must not be projected back into June.

Destination and activation

QUESTION	WHAT STATIC ANALYSIS REACHED	WHAT REMAINS UNRESOLVED
Where?	Native config reads <i>tth</i> host and <i>pat</i> path from XOR/base64/AES-GCM encoded material; TLS port 443.	Plaintext recovery failed authentication; no exact host/path IOC is defensible.
When?	Later Java config exposes <i>navigatorConfig.impulseConfig.enabled</i> ; compiled default is false.	Historical online-config responses, rollout cohort, device conditions and actual request rate.
What?	Later observation-derived data reaches the native request body after transformation.	Exact retained fields, wire bytes, receiver and any user targeting.

EVIDENCE BOUNDARY

This is an unresolved native data path, not a proven exfiltration incident. No caller from *PUSH_LOGGER* or the new pull dispatcher to *Impulse* was found. A controlled network trace and saved online-config response would decide what the code did in use.

Anchors: **14.10.2.3** APK 90499f420b9288d8574e9bf9ae01ee90b2dc1dabb3005877b8655468c204a698; **14.13.0.6** archived base 24e8c96780668a13d7d3fb9780a0c52ccd1012f4ba06515d2f523506a98e5a0a. The inspected native libraries have distinct SHA-256 values retained in the case notes; the PDF records the APK anchors and the exact JNI signature change.

Location updates gain a radio POST

The new path is controlled by app configuration and starts from map/navigation updates, not the reviewed FCM handler.

The Bazaar [ir.balad 4.79.0](#) sample lacks the reviewed [RadioObservationUploader](#) path. In [4.81.1](#), [LocationLogger](#) calls that uploader from map and navigation location updates. The master [isGeoSubmitEnabled](#) flag comes from an AppMeta gRPC response; an absent config defaults it to false. The method accepts non-mock GPS/fused fixes with reported accuracy of 1-50 metres, rate-limits to about one upload per 30 seconds, then builds a [BaladRadioObservation](#) for POST <https://location.raah.ir/v2/geosubmit>. [07]

4.79.0 / SAMPLED

map/nav fix -> location log
no RadioObservationUploader call
no v2/geosubmit literal

4.81.1 / SAMPLED

map/nav fix -> uploader.p(...)
if !isGeoSubmitEnabled: return
if trustedFix && throttle: POST

Reduced from the decompiled [LocationLogger.kt](#) and [RadioObservationUploader.kt](#) paths. The earlier app may have other telemetry.

What the observation model can hold

GROUP	MODEL FIELDS	INTERPRETATION
Position	Latitude, longitude, accuracy, provider, optional speed and timestamp.	A precise fix if the gate and trust checks pass.
Radio	Nearby cell identifiers/MCC/MNC/signal plus Wi-Fi MAC/BSSID/SSID, signal and frequency.	Some Java fields are transient ; exact serialized fields need a wire sample.
Link/context	SHA-256 of a persisted generated app UUID; map or navigation context and spoofing state.	Persistent app-local identifier; not shown to join identities across apps.

Sampled [4.83.0](#) adds a GNSS integrity verdict that can suppress spoofed fixes. By [4.84.0](#), a second remote flag, [useBaladLocations](#), permits Balad's network-location source and batching of up to ten observations. The master geosubmit flag still gates the route. Location and Wi-Fi permissions were already present before [4.81.1](#); the behavior changed without a new location grant. [07]

WHAT WOULD SETTLE HISTORICAL USE

Recover AppMeta [baladNetworkProvider](#) values by time and cohort, the signed build inputs for missing 4.78.x/[4.80.0/4.81.0](#) files, and geosubmit access/body logs. No actual request, recipient ownership proof or attacker-controlled config was recovered.

Exact transition: [4.79.0 4c015bc86454142d7d5e4318e02454d0e1c7c2c9c602f3227fbde5a631c31784](#); [4.81.1 da7e348ba2fd161e0a0d05b7ad7a216f95389f9e69219144d5cc72bf7454370b](#) (VT first submission 10 June). Bazaar and Play packages/signers are analyzed as separate tracks.

A gated radio snapshot in the driver app

The new request is distinct from Snapp's existing driver location publishing.

The acquired driver 5.13.0 and 5.15.0 DEX lack `RadioSignalCollectionRequest`, `isWifiCollectionEnabled` and the reviewed `publishRadioSignalData` path. Both acquired 5.17.0 variants and 5.18.0 contain it. The first positive file reached VT on 26 July; unsampled 5.16.0 prevents an exact introduction claim. [10]

In 5.17.0, `JSDetectorInteractor` requires the AB flag, a positive collection interval and a positive Wi-Fi or cell sample count. The request builder combines fused/network/manually selected/suggested locations, Wi-Fi scans, connected cells, foreground state, ride status and device time. It POSTs to the default

<https://locations.snapp.site/v1/driver/collect>, but a server-provided location base can replace that default. The code does not show the effective historical flags or destination. [10]

SELECTED DECOMPILED REQUEST FIELDS / 5.17.0

```
@SerializedName("device_timestamp") String timestamp;
@SerializedName("location") LocationSources location;
@SerializedName("wifi_scans") List<WifiScan> wifi;
@SerializedName("connected_cells") List<ConnectedCell> cell;
@SerializedName("app_state") int foregroundState;
@SerializedName("ride_status") String rideStatus;
```

Field names come from the preserved JADX request class; variable names above are shortened for legibility. This is not a captured POST body.

Wi-Fi BSSID/SSID values are hashed with unsalted SHA-256 in the request model, while coordinates and cell identifiers remain structurally available. Those hashes do not erase a radio-location fingerprint. The familiar `ACCESS_BACKGROUND_LOCATION`, fine/coarse location, Wi-Fi-state and foreground-location declarations were already present at the 5.15.0 checkpoint. [10]

SAMPLED CHECKPOINT	KNOWN COLLECTOR PATH	DATE BASIS
5.15.0	Reviewed route absent from raw DEX.	VT 13 April 2026
5.17.0 / two variants	Gate, DTO and POST present in both; relevant DEX is identical.	VT 26 July / 4 August
5.18.0	Collector retained; process-death location receiver added.	VT 5 September

ALTERNATIVE EXPLANATION

The path is compatible with positioning, signal quality or spoofing work. No attacker-controlled endpoint, enabled AB cohort, historical request or malicious use is verified. Recover the AB config, dynamic base URL and `/collect` receipts before treating it as an incident.

Polling and updates are separate

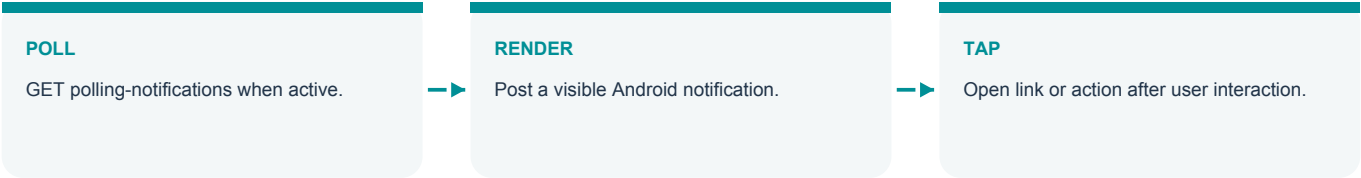
A notification inbox and an in-app updater deserve review, but neither is the Divar receiver shortcut in these samples.

Passenger: Fanoos polling

Passenger 8.37.0 and sampled 8.38.0 lack `cab.snapp.fanoos`. Acquired 8.39.0 and later contain `GET v1/passenger/polling-notifications`, with `notifications[].{id,data,type}` and an `x-poll-interval` response header. The reviewed default renderer posts a visible Android notification; its link/action path follows a user tap. I found no `Fanoos.ignite()` callsite in decompiled passenger 8.39.0, so fresh-install activation remains unverified. [10]

The archive dates are especially easy to overread: Uptodown lists the **version 8.39.0** on 25 February, while the preserved APK came from APKPure's 2 March listing. This record cannot put that exact binary on phones before the 28 February war boundary. Driver Fanoos polling is present by sampled 5.15.0, with authorization and feature checks around its `ignite()` call.

REVIEWED FANOOS DEFAULT BEHAVIOR



Behrooz: an update channel, not a recovered stage

The driver app also includes a Behrooz updater that can read `UpdateConfig.directUrl`, download an APK and ask Android `PackageInstaller` or an install intent to apply it. Install permissions vary within the same 5.17.0 version: one acquired variant declares `REQUEST_INSTALL_PACKAGES` and `UPDATE_PACKAGES_WITHOUT_USER_ACTION`, the other does not. Passenger variants also differ. A compromised update configuration could matter, but no hostile URL, APK or install outcome was recovered. The inspected FCM and polling handlers do not demonstrate a direct loader edge to Behrooz.

MECHANISM	OBSERVED PATH	IMPORTANT LIMIT
Fanoos passenger	Server poll -> visible notification -> tap action.	Exact 8.39 timing and activation unresolved.
Fanoos driver	Authorized/flagged polling inbox by 5.15.0.	No silent command-loader branch shown.
Behrooz	Configured APK URL -> download -> installer.	No attacker patch or install trace; variant permissions differ.

COMPARISON RULE

The mere presence of polling, push SDKs, an updater or package-install permission is not a finding of the Divar backdoor. The decisive question is which received bytes reach a worker, under what gate, and whether a real server supplied them.

Tapsi production geolocation request

The inspected callsites POST radio and prior-location observations to Tapsi production APIs and use the response as a location estimate.

The acquired Tapsi corpus begins in June for passengers and July for drivers; it does not cover January-May. Passenger 7.7.0 has the route string by its 12 June VT submission. Decompiled 7.8.4 and 7.10.0 both call `POST v3/geolocation/geolocate` on the production `https://p1.tapsi.ir/api/` base with the same seven serialized fields. The later deobfuscation does not represent a new payload. The repository requires at least one cell or Wi-Fi observation before sending. [11]

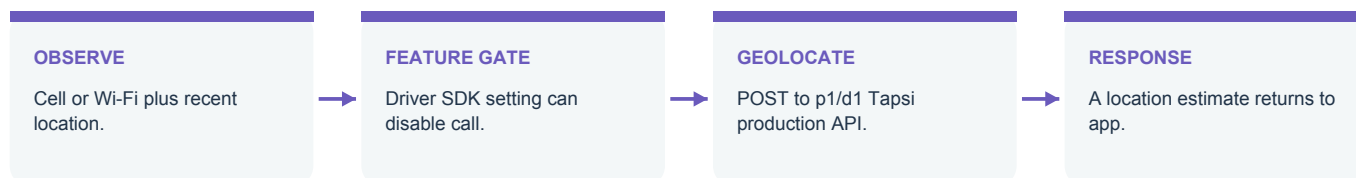
PASSENGER REQUEST / SEVEN STABLE JSON FIELDS

```
cellTowers, wifiAccessPoints,
cellTowersRepeatCount, wifiAccessPointsRepeatCount,
isWifiEnabled, lastLocation, lastInHouseNetworkLocation
POST https://p1.tapsi.ir/api/v3/geolocation/geolocate
```

Both 7.8.4 and 7.10.0 expose this contract in decompiled DTO/API methods; this is not a captured request.

The inspected driver 8.8.0 landing app builds the analogous request from observation flows and recent driver locations, POSTs through `https://d1.tapsi.ir/api/`, and reads the location estimate. The inspected `EnabledFeatures` gate and SDK config default are disabled; live server settings are unknown. A route string in a lower version is a discovery lead, not proof of execution. [11]

TAPSI POSITIONING PATH



No non-Tapsi recipient, actual off-band exfiltration receipt, affected account or operator control was identified. This is sensitive location data in an apparent first-party positioning service. The first VT submission of a positive sample does not date the code's first public release.

VERSION-ORDER TRAP

The sampled driver 7.13.6 negative checkpoint was submitted to VT after a higher-version positive. For that pair, **version order** bounds the code change; VT order cannot be used as a developer rollout sequence. Missing January-May bytes preclude a pre-war verdict.

Two signed 8.8.0 flavors diverge

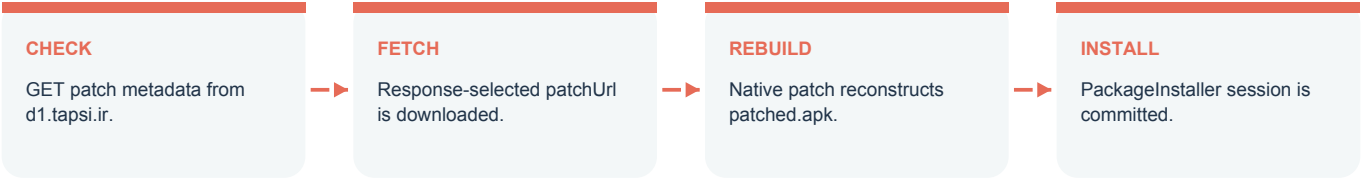
The landing build has a binary-patch updater. The same-version Myket build lacks it; the difference is a distribution flavor.

Both driver **8.8.0** APKs use package `taxi.tap30.driver`, versionCode 1080080000 and the same signer certificate. The Myket file identifies itself as `myket_productionFinalRelease`; the landing file says `landing_productionFinalLanding`. The landing build adds Yadegar patch classes, `libapkpatch.so`, install-result handling and install/update manifest declarations. This is not an observed insertion followed by retraction. [11]

EXACT 8.8.0 FILE	MODULE STATE	VT FIRST SUBMISSION
Myket / 4f26428e71051bcf116cc97d33670adf76f24cafe8b1e1a550e19eb42a94ee47	No Yadegar patch module in reviewed variant.	5 September 2026
Landing / 1647b9af56b3923ac8a32c3ee9aeb9513563d73a86181f1db05a10f3bbbed95a4	Yadegar patch path and install declarations.	14 September 2026

What the landing updater does

YADEGAR LANDING PATH



The server response contains `hasNewUpdate`, `patchUrl`, `patchName`, `targetVersion` and `patchSize`. The inspected Java/Kotlin path does not compare an independent expected digest or detached signature before the installer commit; Android package-install and signer checks still apply. A server-selected update URL is a security-relevant delivery surface, but no malicious patch, attacker control or successful install was recovered. No edge from the inspected push handler to Yadegar was shown.

WHAT WOULD VERIFY THIS ROUTE

Preserve Yadegar configuration and `get-patch` response logs, downloaded patch bytes, patch-host resolution, reconstructed APK hash and installer result. Compare each exact artifact with developer build approval and store-channel records.

The same signer is useful for grouping the pair but cannot establish who authorized it. A compromised signing key or release pipeline could produce a valid signature; neither is proven here.

What changed, and what did not

These are the meaningful behavior boundaries in acquired files. Dates are observation dates unless explicitly marked as archives.

TRANSITION	NEW OR PERSISTENT BEHAVIOR	CONSEQUENCE / LIMIT
Neshan 14.8.9 -> 14.8.10	App-open pull joins an existing FCM command dispatcher; <code>PUSH_LOGGER</code> remains the same command.	A silent server response can reach file upload; no response or receipt recovered.
Neshan 14.10.2.3 -> 14.13.0.6	Native HTTPS path evolves to accept Java observation JSON and send transformed, input-derived body.	Later GPS input is proven at code level; June fields and plaintext host remain unknown.
Balad 4.79.0 -> 4.81.1	Location callbacks gain config-gated radio observation POST.	Sensitive collection capability; earlier gap and live flag values unresolved.
Balad 4.83.0 -> 4.84.0	Second flag permits Balad network-location source and batches up to ten.	Potentially changes source and volume; master geosubmit gate persists.
Snapp driver 5.15.0 -> 5.17.0	AB-gated location/radio request and <code>collect</code> POST appear.	No 5.16.0 bytes; default host can be replaced by dynamic base.
Snapp passenger 8.38.0 -> 8.39.0	Fanoos poll classes appear; default renderer posts visible notification.	Preserved hash is not placed before war by version-only archive listing.
Tapsi passenger 7.8.4 -> 7.10.0	Same seven-field geolocation request survives deobfuscation.	Class-name change alone is not a new upload.
Tapsi driver 8.8.0 Myket / landing	Landing flavor alone carries Yadegar patch updater and install declarations.	Same version/channel variant; no malicious patch or insertion/retraction shown.

COMPARISON TO DIVAR

Divar's confirmed sampled code has a special push/response entry that diverts to a fetch, file write and reflective invocation worker. None of the paths above demonstrated that same end-to-end worker chain in these four apps. Different malicious logic, native-only code or missing releases remain outside this narrow negative.

The changes rarely needed a new grant

Manifest comparisons explain app capability, not whether a user granted access or a company authorized an update.

TRACK / BOUNDARY	RELEVANT MANIFEST OBSERVATION	INTERPRETATION
Balad / 4.79.0 -> 4.81.1	Coarse/fine location, Wi-Fi state and Internet were already declared. Bazaar referrer/attribution changes appear separately.	Uploader can use existing grants; no new location prompt proves nothing about use.
Neshan / 14.8.9 -> 14.8.10	No new permission. FCM service stays non-exported and registered for Firebase messaging.	New pull reaches old file-upload command through app code and its existing access.
Snapp driver / 5.15.0 -> 5.17.0	Background/fine/coarse location, Wi-Fi state and foreground-location permissions pre-exist.	Radio collector is a code/config delta, not a manifest delta.
Tapsi driver / 8.8.0 same version	Landing flavor adds install/update declarations and install-result receiver; Myket flavor lacks them.	Distribution choice changes update capability; it is not proof of compromise.

What signing establishes

Every acquired Balad, Neshan, Snapp and Tapsi APK has an exact hash and an associated signer fingerprint in the companion inventory. Balad's Bazaar and Play packages are distinct signing tracks; Snapp passenger and driver are distinct products; Tapsi passenger and driver are separate packages. The Neshan variants share their compared lineage. These groupings make adjacent diffs more meaningful, but **a valid signature is not an authorization verdict**. A stolen key, unauthorized source edit or compromised build pipeline can still produce signed bytes. [07-11]

Code, deployment and traffic are three different records

- **Code:** a caller reaches a request or worker under stated conditions in the exact acquired file.
- **Deployment:** a store, CDN or device installed that exact hash for a known cohort.
- **Traffic:** a retained server/device record shows the condition passed and data or patch moved.

BOUNDARY

This study establishes selected code capabilities and exact-file observations. It has no production backend configuration, OneSignal/FCM send record, device network trace, Tapsi patch payload, Neshan command response or location upload receipt. Consequently it does not estimate victims or attribute the paths to a threat actor.

Records that could change the verdict

Ask for exact-hash deployment and route-specific receipts before attempting a cross-app narrative.

PRIORITY	PRESERVE AND CORRELATE	QUESTION ANSWERED
01 / build custody	CI source commit and approval, signing event, exact output hash, store/CDN channel and device rollout cohort for each boundary and same-version variant.	Was the changed code an authorized build, and where did that exact file go?
02 / Neshan command	FCM payloads, <code>/notification/pull</code> responses and access logs, <code>PUSH_LOGGER</code> dispatcher records, <code>/logger/uploadLogFile</code> bodies/receipts; especially <code>sendInfo</code> and <code>fileList</code> .	Did a silent diagnostic request select unusual files or users?
03 / Neshan native	Saved <code>online-config/config/v5.2</code> responses, <code>impulseConfig</code> values, controlled app traffic, TLS destination and native request bytes for the two JNI generations.	Which host and fields were used, and was it enabled?
04 / Balad	AppMeta <code>baladNetworkProvider</code> history by cohort; <code>isGeoSubmitEnabled</code> and <code>useBaladLocations</code> ; <code>/v2/geosubmit</code> request and receipt logs.	Which devices uploaded radio/location observations, if any?
05 / Snapp	Driver AB <code>isWifiCollectionEnabled</code> , counts/intervals, dynamic location base and <code>/collect</code> receipts; passenger Fanoos flags/poll records and device notification history.	Was collection or polling active, and to what endpoint?
06 / Tapsi	Geolocate feature flags and request/response logs; Yadegar <code>get-patch</code> responses, downloaded patch, reconstructed APK hash and installer result.	Was a patch or location request actually delivered?

The cross-app join test

Only after proving individual events should investigators test whether the same device or person appears in a location record and a delivery target. Use lawful, independently verified device/account identifiers and exact timestamps. A hashed Balad app UUID is not shown to equal a Divar, Snapp, Neshan or Tapsi identifier. A common time window, country or valid signer does not establish common control.

PRESERVATION FIRST

Push bodies can be absent from ordinary notification history when a handler returns before posting UI. Preserve backend sends, pull responses and upload receipts while retention permits. For Divar's OneSignal/FCM route, consult the companion case file's separate handoff; the app providers here must be assessed on their own code and logs.

Files behind the pivotal comparisons

Each SHA-256 is clickable to its VirusTotal file page. A file page or host name is an evidence pivot, not a malicious IOC.

BALAD / 4.79.0 / Bazaar	uploader absent
4c015bc86454142d7d5e4318e02454d0e1c7c2c9c602f3227fbde5a631c31784	
BALAD / 4.81.1 / Bazaar	geosubmit present
da7e348ba2fd161e0a0d05b7ad7a216f95389f9e69219144d5cc72bf7454370b	
NESHAN / 14.8.9	pull absent
681b4f975d40b9d0f1dfa9d5d6b79d59cbd3ca46746a6f9f013ef1e1bf7bcfd4	
NESHAN / 14.8.10	pull added
947c4a9412ef577b29c87d87764b4e9e0e3ddfd1966bff2cbab83adb3f2f8115	
NESHAN / 14.10.2.3	Impulse native path
90499f420b9288d8574e9bf9ae01ee90b2dc1dabb3005877b8655468c204a698	
NESHAN / 14.13.0.6	GPS input to native POST
24e8c96780668a13d7d3fb9780a0c52ccd1012f4ba06515d2f523506a98e5a0a	
SNAPP DRIVER / 5.15.0	collector absent
74660538613dd5ccb077ed46a3124e9242258e1c139520642d5f1599613cbad4	
SNAPP DRIVER / 5.17.0	collector present
2c7bee97b304134c58aafc15d9935a55a9091488e2a200080353218932bfe671	
TAPSI PASSENGER / 7.8.4	geolocate call
e7e7597fcc8e2dc93535bd8eebd862433da34c1c67a1c73120e3268a84ae2869	
TAPSI DRIVER / 8.8.0 / Myket	no Yadegar
4f26428e71051bcf116cc97d33670adf76f24cafe8b1e1a550e19eb42a94ee47	
TAPSI DRIVER / 8.8.0 / landing	Yadegar present
1647b9af56b3923ac8a32c3ee9aeb9513563d73a86181f1db05a10f3bbbed95a4	

The [full 103-APK inventory](#) preserves all inspected hashes, package tracks, versionCodes, signing-fingerprint algorithm, acquisition source and exact date basis. This page selects only files that make a substantive boundary or variant legible.

How to read and reproduce this case

The primary evidence is the acquired APK bytes. Public pages below date or locate individual files; none supplies missing backend traffic.

- [01] [Divar case notes by Ramin Farajpour Cami](#)
- [02] [My Divar disclosure on X](#)
- [03] [Divar technical case file v2.7](#)
- [04] [UN: opening strikes, 28 February 2026](#)
- [05] [UN: 7 April two-week ceasefire announcement](#)
- [06] [UN: June peace deal and intermittent fire](#)
- [07] [Balad 4.81.1 exact-file checkpoint](#)
- [08] [Neshan pull checkpoint](#)
- [09] [Neshan native checkpoint](#)
- [10] [Snapp driver 5.17.0 exact-file checkpoint](#)
- [11] [Tapsi driver 8.8.0 landing exact file](#)
- [12] [Full normalized sample inventory](#)
- [13] [Balad archive version listings](#)
- [14] [Snapp APKPure passenger archive](#)
- [15] [Snapp Uptodown version archive](#)
- [16] [Tapsi passenger 7.8.4 exact file](#)

Technical evidence notes

The public [evidence index](#) links the inspected [Balad](#), [Neshan](#), [Snapp](#) and [Tapsi](#) method/line anchors and their exact APK hashes. The notes are decompiled/static evidence, not server telemetry.

Method and limits

For each acquired file I recorded the SHA-256, package, versionCode and signer fingerprint, then compared adjacent DEX, manifest and relevant decompiled methods. Distinctive Divar trigger/worker checks served as triage, followed by path-specific review of command dispatch, location callbacks, network interfaces, native call flow and updaters. Obfuscation and JADX errors were checked against raw DEX or repeated variants where material; the Impulse conclusion is bounded by disassembled request-path reachability. This was **static analysis**, not an observed victim-device run.

REPORT HISTORY

Version 1.2 was a chronology-led preliminary cross-app review. Version 2.0 follows the Divar v2.7 case-file design, centers the four additional apps, adds 103-file coverage and semantic path diagrams, and sharpens date, native-host and server-use limits. Prepared 5 October 2026 by Hamid Kashfi with LLM assistance.